

Cell 1: Setup & Upload

```

# =====
# FIXED: Robust CSV Loading
# =====

from google.colab import files
import pandas as pd
import io

print("📁 Please re-upload 'marketing_and_sales_data_evaluate_lr.csv'")
uploaded = files.upload()

filename = list(uploaded.keys())[0]

# Try multiple ways to load the problematic CSV
try:
    # Standard read
    df = pd.read_csv(filename)
    print("✅ Loaded with default settings")
except Exception as e:
    print("Standard read failed, trying robust methods...")

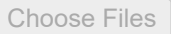
    # Method 1: Skip bad lines
    try:
        df = pd.read_csv(filename, on_bad_lines='skip')
        print("✅ Loaded with on_bad_lines='skip'")
    except:
        # Method 2: Use Python engine (more flexible)
        try:
            df = pd.read_csv(filename, engine='python', sep=',', quotechar='"', on_
            print("✅ Loaded with Python engine")
        except:
            # Method 3: Read as text and inspect
            print("Trying manual inspection...")
            with open(filename, 'r', encoding='utf-8') as f:
                lines = f.readlines()[:10]
                for i, line in enumerate(lines):
                    print(f"Line {i+1}: {len(line.split(','))} fields → {line[:100]}")

            # Final attempt with more options
            df = pd.read_csv(filename, engine='python', sep=',', error_bad_lines=False)

print(f"\n✅ Final Shape: {df.shape}")
print(df.head())
print("\nColumns:", df.columns.tolist())

```

 Please re-upload 'marketing_and_sales_data_evaluate_lr.csv'

 marketing_...aluate_lr.csv

marketing_and_sales_data_evaluate_lr.csv(text/csv) - 181277 bytes, last modified: 6/15/2026 - 100% done

Saving marketing_and_sales_data_evaluate_lr.csv to marketing_and_sales_data_evaluate

 Loaded with default settings

 Final Shape: (4572, 4)

	TV	Radio	Social_Media	Sales
0	16.0	6.566231	2.907983	54.732757
1	13.0	9.237765	2.409567	46.677897
2	41.0	15.886446	2.913410	150.177829
3	83.0	30.020028	6.922304	298.246340
4	15.0	8.437408	1.405998	56.594181

Columns: ['TV', 'Radio', 'Social_Media', 'Sales']

```
# Data Quality Check
print("=== Data Info ===")
print(df.info())
print("\n=== Missing Values ===")
print(df.isna().sum())

# Clean the data
df_clean = df.dropna(subset=['Sales']).copy()
df_clean = df_clean.dropna()

print(f"\nCleaned Shape: {df_clean.shape}")
```

```
=== Data Info ===
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4572 entries, 0 to 4571
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  -
0    TV              4562 non-null   float64
1    Radio           4568 non-null   float64
2    Social_Media    4566 non-null   float64
3    Sales           4566 non-null   float64
dtypes: float64(4)
memory usage: 143.0 KB
None

=== Missing Values ===
TV              10
Radio           4
Social_Media    6
Sales           6
dtype: int64

Cleaned Shape: (4546, 4)
```

Cell 2: Data Exploration & Cleaning

```

print("=== Dataset Information ===")
print(df.info())
print("\n=== Missing Values ===")
print(df.isna().sum())

# Drop rows where Sales is missing (as required)
df_clean = df.dropna(subset=['Sales']).copy()

# Drop any remaining missing values
df_clean = df_clean.dropna()

print(f"\nOriginal shape: {df.shape}")
print(f"Cleaned shape: {df_clean.shape}")
print("Missing values after cleaning:", df_clean.isna().sum().sum())

```

```

=== Dataset Information ===
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4572 entries, 0 to 4571
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  -
0   TV               4562 non-null   float64
1   Radio            4568 non-null   float64
2   Social_Media     4566 non-null   float64
3   Sales            4566 non-null   float64
dtypes: float64(4)
memory usage: 143.0 KB
None

```

```

=== Missing Values ===
TV           10
Radio        4
Social_Media 6
Sales        6
dtype: int64

```

```

Original shape: (4572, 4)
Cleaned shape: (4546, 4)
Missing values after cleaning: 0

```

Cell 3: Exploratory Data Analysis (EDA)

```

# Pairplot
plt.figure(figsize=(10, 8))
sns.pairplot(df_clean, diag_kind='kde')
plt.suptitle("Pairplot: Marketing Spend vs Sales", y=1.02)
plt.show()

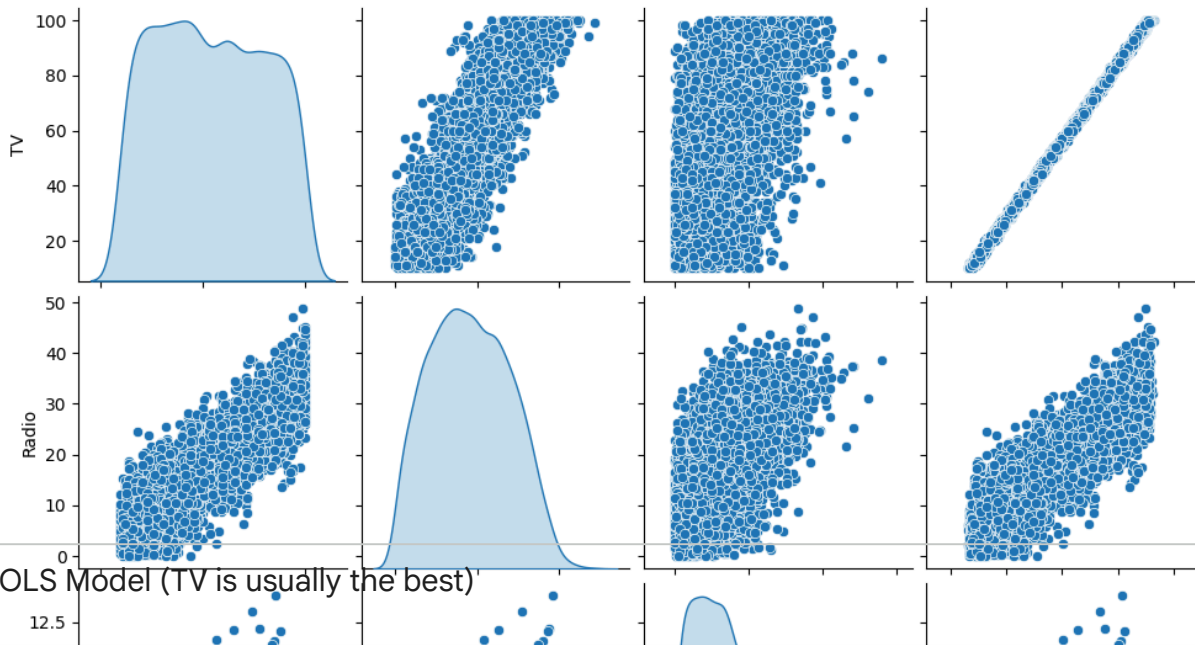
# Correlation heatmap
plt.figure(figsize=(8, 6))
corr = df_clean.corr()
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt='.3f')
plt.title("Correlation Matrix")
plt.show()

```

```
# Strongest predictor
correlations = df_clean.corr()['Sales'].drop('Sales').sort_values(ascending=False)
print("🔥 Correlations with Sales:")
print(correlations)
```


<Figure size 1000x800 with 0 Axes>

Pairplot: Marketing Spend vs Sales

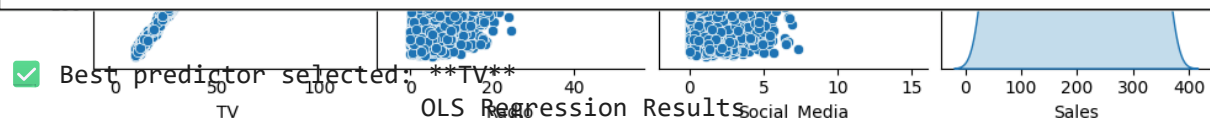


```
best_predictor = correlations.index[0]
print(f"\n✅ Best predictor selected: **{best_predictor}**")

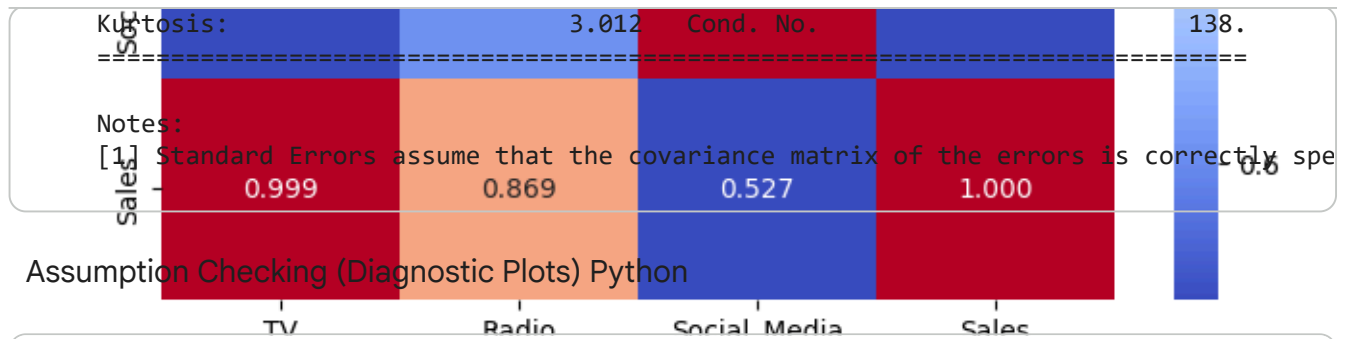
# Prepare data
ols_data = df_clean[[best_predictor, 'Sales']]

# Fit model
model = ols(formula=f'Sales ~ {best_predictor}', data=ols_data).fit()

# Show summary
print(model.summary())
```



Correlation Matrix					
Dep. Variable:	Sales				R-squared: 0.999
Model:	OLS				Adj. R-squared: 0.999
Method:	Least Squares				F-statistic: 4.517e+06
Date:	Sun, 21 Jun 2026				Prob (F-statistic): 0.00
Time:	12:34:51				Log-Likelihood: -11366.
No. Observations:	4546				AIC: 2.274e+04
Df Residuals:	4544				BIC: 2.275e+04
Df Model:	1				
Covariance Type:	nonrobust				
	coef	std err	t	P> t	[0.025 0.975]
Intercept	-0.1325	0.101	-1.317	0.188	-0.330 0.065
TV	3.5615	0.002	2125.272	0.000	3.558 3.565
Radio	0.869	0.006	146.667	0.000	0.857 0.881
Social Media	0.528	0.001	528.000	0.000	0.526 0.530
Omnibus:	0.052				Durbin-Watson: 1.998
Prob(Omnibus):	0.974				Jarque-Bera (JB): 0.031
Skewness:	0.606				Prob(JB): 0.985
Kurtosis:	1.000				Prob(KS): 0.7



```
fitted = model.fittedvalues
residuals = model.resid

plt.figure(figsize=(14, 10))

# 1. Residuals vs Fitted (Linearity + Homoscedasticity)
plt.subplot(2, 2, 1)
sns.scatterplot(x=fitted, y=residuals)
plt.axhline(y=0, color='r', linestyle='--')
plt.xlabel('Fitted Values')
plt.ylabel('Residuals')
plt.title('1. Residuals vs Fitted')

# 2. Q-Q Plot (Normality)
plt.subplot(2, 2, 2)
sm.qqplot(residuals, line='s', ax=plt.gca())
plt.title('2. Q-Q Plot (Normality)')

# 3. Histogram of Residuals
plt.subplot(2, 2, 3)
sns.histplot(residuals, kde=True)
plt.title('3. Residuals Distribution')

# 4. Scale-Location Plot (Homoscedasticity)
plt.subplot(2, 2, 4)
sns.scatterplot(x=fitted, y=np.sqrt(np.abs(residuals)))
plt.axhline(y=0, color='r', linestyle='--')
plt.xlabel('Fitted Values')
plt.ylabel('√|Residuals|')
plt.title('4. Scale-Location Plot')

plt.tight_layout()
plt.show()
```

